SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1737

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 3 **Duration: 1 Hour**
Total Marks: 30

Annexure A

Dry Run Evaluation

Code of Conduct:

*I **Saai Vardhan D / 192525205** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Saai Vardhan D

Annexure A

Dry Run Evaluation

Name: Saai Vardhan D | Reg. No.: 192525205 | Course: CSA1737 – Artificial Intelligence

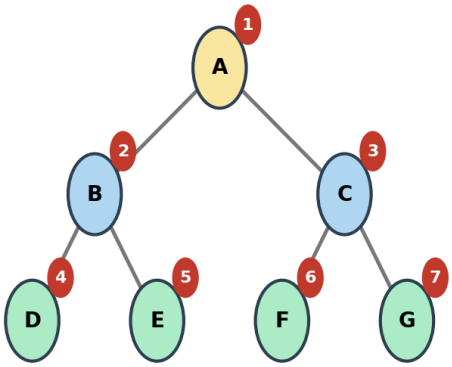
Question 1 – Breadth-First Search (BFS) (10 Marks)

Graph given: A is the root; A has children B and C; B has children D and E; C has children F and G. (Fig. 1 shows the graph.)

Task: Starting from node A, perform a Breadth-First Search and complete the table.

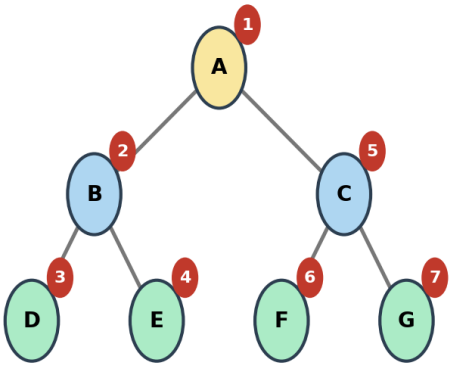
Fig. 1 Graph traversal (red number = order of visit)

(a) BFS – level by level (queue, FIFO)



Visit order: A → B → C → D → E → F → G

(b) DFS – deepest first (stack, LIFO)



Visit order: A → B → D → E → C → F → G

Method

BFS uses a **queue (FIFO – First In, First Out)**. Steps: (1) put the start node in the queue; (2) remove the node at the front, mark it visited; (3) add all its unvisited children to the rear of the queue (left to right); (4) repeat until the queue is empty. Because nodes are added at the rear and removed from the front, all nodes at depth d are visited before any node at depth d + 1 – so the graph is explored **level by level**.

Dry-run table

Iteration	Queue at start (front → rear)	Visited node (removed from front)	Queue after adding children
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]
5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[ ] (empty – stop)

Answers to the questions

1. BFS traversal order: A → B → C → D → E → F → G (Level 0: A; Level 1: B, C; Level 2: D, E, F, G).

2. Queue contents after each iteration: [B, C] → [C, D, E] → [D, E, F, G] → [E, F, G] → [F, G] → [G] → [ ].

Observations: the queue never contains more than 4 nodes (the width of the widest level); every node is visited exactly once (7 nodes in 7 iterations); time complexity O(V + E) = O(7 + 6) here; BFS is complete and finds the shallowest node.

Question 2 – Depth-First Search (DFS) (10 Marks)

Task: Using the same graph, perform DFS starting from A and complete the table.

Method

DFS uses a **stack (LIFO – Last In, First Out)**. Steps: (1) push the start node; (2) pop the top node and mark it visited; (3) push its unvisited children so that the **left-most child is on top** (i.e. push the right child first, then the left child); (4) repeat until the stack is empty. The search follows one branch as deep as possible before backing up to the next unexplored branch.

Dry-run table (stack written bottom → top; the right-most element is the top)

Iteration	Stack at start	Visited node (popped from top)	Stack after pushing children
1	[A]	A	[C, B] (B on top)
2	[C, B]	B	[C, E, D] (D on top)
3	[C, E, D]	D	[C, E] (D is a leaf – nothing pushed)
4	[C, E]	E	[C] (leaf)
5	[C]	C	[G, F] (F on top)
6	[G, F]	F	[G] (leaf)
7	[G]	G	[ ] (empty – stop)

DFS traversal order: A → B → D → E → C → F → G (Fig. 1(b)).

BFS vs DFS on this graph

Feature	BFS	DFS
Data structure	Queue (FIFO)	Stack (LIFO) / recursion
Visit order	A B C D E F G (level by level)	A B D E C F G (branch by branch)
Maximum frontier size	4 nodes ($O(b^d)$)	3 nodes ($O(b \cdot d)$)
Shortest path guaranteed (unweighted)?	Yes	No
Complete?	Yes	Only for finite graphs / with a visited set
Time complexity	$O(b^d) \approx O(V + E)$	$O(b^m) \approx O(V + E)$
Space complexity	$O(b^d)$ – large	$O(b \cdot m)$ – small

Both visit all 7 nodes exactly once. DFS needs less memory, whereas BFS guarantees that the first time a node is reached it is by the shortest path.

Question 3 – BFS on the 8-Puzzle (10 Marks)

Initial state:

1	3	6
5	0	2
4	7	8

Goal state:

1	2	3
4	5	6
7	8	0

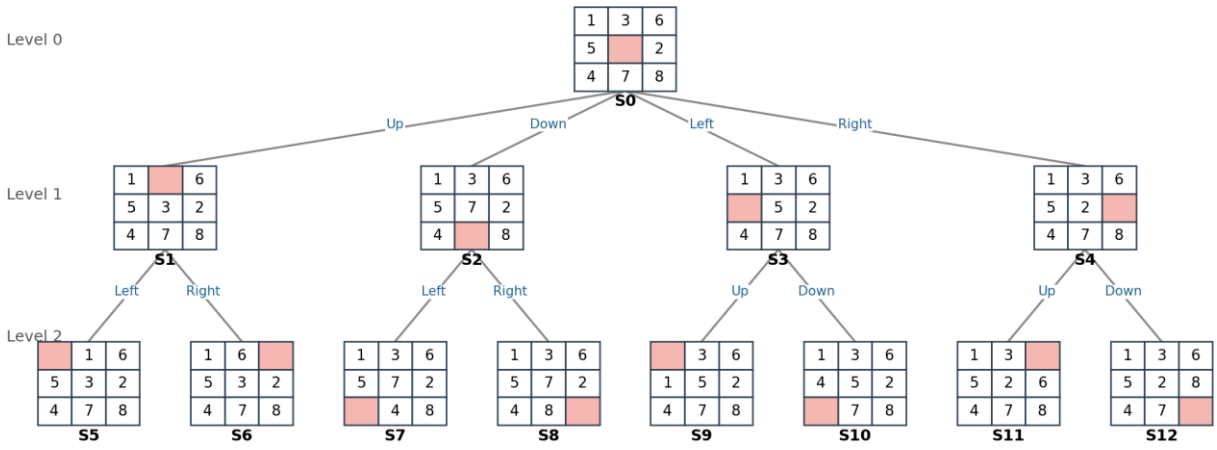
(0 denotes the blank.) **Task:** trace BFS until the goal state is reached and fill in the table.

Problem formulation and BFS settings

Element	Definition
State	An arrangement of the tiles 1–8 and the blank on the 3 × 3 board.
Actions	Slide the blank Up, Down, Left or Right (if the move stays on the board). Branching factor $b \leq 4$.
Step cost	1 for every move (unweighted).
Operator order	Up, Down, Left, Right – children are generated in this order and appended to the rear of the queue (FIFO).
Repeated states	A state that has already been generated is not added again (visited set), so the blank never simply undoes the previous move.
Goal test	Performed when a state is removed from the queue for expansion.

To keep the table readable, the states are named S0, S1, S2 … in the order in which they are generated (S0 = initial state). The boards for S0–S12 are drawn in Fig. 2.

Fig. 2 BFS state-space tree for the first 5 iterations (blank shown in pink; Sk = k-th generated state)



Dry-run table (first five iterations)

Iteration	Current state (expanded)	Queue before expansion (front → rear)	Queue after expansion
1	S0 (initial)	[S0]	[S1, S2, S3, S4]
2	S1 (blank moved Up)	[S1, S2, S3, S4]	[S2, S3, S4, S5, S6]
3	S2 (blank moved Down)	[S2, S3, S4, S5, S6]	[S3, S4, S5, S6, S7, S8]
4	S3 (blank moved Left)	[S3, S4, S5, S6, S7, S8]	[S4, S5, S6, S7, S8, S9, S10]
5	S4 (blank moved Right)	[S4, S5, S6, S7, S8, S9, S10]	[S5, S6, S7, S8, S9, S10, S11, S12]

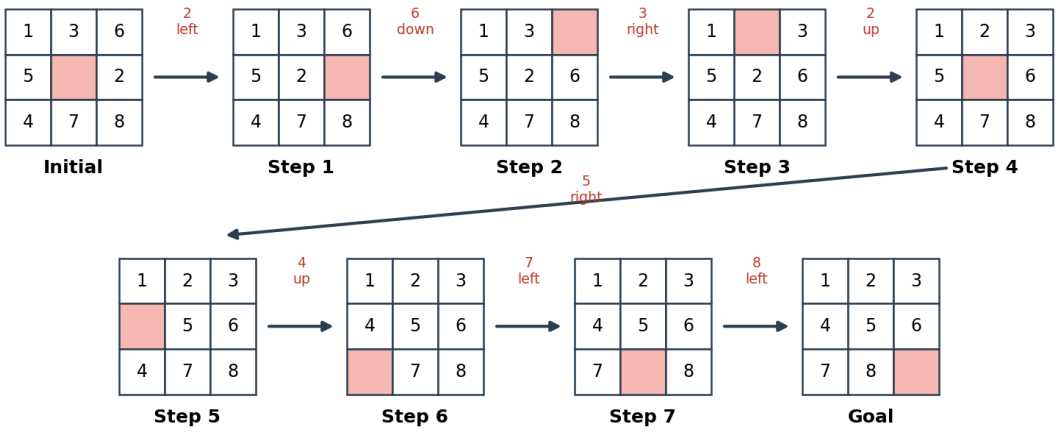
State	Board (row by row)	Generated from	State	Board (row by row)	Generated from
S0	136 / 5_2 / 478	initial	S7	136 / 572 / _48	S2, Left
S1	1_6 / 532 / 478	S0, Up	S8	136 / 572 / 48_	S2, Right
S2	136 / 572 / 4_8	S0, Down	S9	_36 / 152 / 478	S3, Up
S3	136 / _52 / 478	S0, Left	S10	136 / 452 / _78	S3, Down
S4	136 / 52_ / 478	S0, Right	S11	13_ / 526 / 478	S4, Up
S5	_16 / 532 / 478	S1, Left	S12	136 / 528 / 47_	S4, Down
S6	16_ / 532 / 478	S1, Right			

("_" marks the blank.) For each expanded state, the move that would return to its parent is not generated again, e.g. S1 has only Left and Right children because Down leads back to S0.

Important note: the shortest solution is **8 moves long**, so the goal is not reached within the first five iterations. BFS must finish expanding every state at depth 7 and then reach the goal at depth 8. The complete progress of the search is summarised below (it is the same dry run, continued).

Depth (level)	States at this depth	Iterations in which they are expanded	Total expanded so far
0	1	1	1
1	4	2 – 5	5
2	8	6 – 13	13
3	8	14 – 21	21
4	16	22 – 37	37
5	32	38 – 69	69
6	60	70 – 129	129
7	72	130 – 201	201
8	136 (goal is one of them)	202 – 311; **goal dequeued in iteration 311**	311

Fig. 3 Optimal solution path found by BFS (8 moves)



Answers to the questions

1. Which node is expanded in each iteration? Iteration 1: S0 (initial state); iteration 2: S1; iteration 3: S2; iteration 4: S3; iteration 5: S4 – i.e. the four children of the initial state in left-to-right (Up, Down, Left, Right) order. Iterations 6 onward expand S5, S6, S7 … in the same first-in-first-out order (all depth-2 states, then all depth-3 states, and so on). The goal state is removed from the queue and recognised in iteration 311.

2. How many states are generated in total? With the operator order above and repeated states discarded, BFS generates **499 new states** (500 distinct states including the initial state) by the time the goal is dequeued. Of these, **311** are expanded (310 non-goal expansions + the goal), and the remaining 189 are still waiting in the queue. (The count depends slightly on operator order and on whether the goal test is made at generation or at expansion; the solution depth, 8, does not.)

3. Complete solution path (blank moves: Right, Up, Left, Down, Left, Down, Right, Right):

Step	Move of the blank (tile moved)	Board after the move
0	Initial state	136 / 5_2 / 478
1	Right (tile 2 slides left)	136 / 52_ / 478
2	Up (tile 6 slides down)	13_ / 526 / 478
3	Left (tile 3 slides right)	1_3 / 526 / 478
4	Down (tile 2 slides up)	123 / 5_6 / 478
5	Left (tile 5 slides right)	123 / _56 / 478
6	Down (tile 4 slides up)	123 / 456 / _78
7	Right (tile 7 slides left)	123 / 456 / 7_8
8	Right (tile 8 slides left)	**123 / 456 / 78_ = GOAL**

The solution has **8 moves** and cost 8: Initial → S4 → S11 → … → Goal, as drawn in Fig. 3.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle?

- **Level-order expansion:** with a FIFO queue all states at depth d are expanded before any state at depth d + 1. So the first time the goal is removed from the queue, no shallower goal can exist.

- **Uniform step cost:** every move costs 1, so path cost = depth. The shallowest goal is therefore the cheapest goal; BFS is a special case of uniform-cost search with all costs equal.
- **Completeness:** the branching factor is finite ($b \leq 4$) and the state space is finite ($9!/2 = 181{,}440$ reachable states), so BFS cannot miss a solution if one exists.
- **Proof sketch:** suppose a solution of length $k < 8$ existed. Its parent states would be at depths $0 \ldots k - 1$ and would be expanded before depth 8, so the goal would have been generated and dequeued earlier – contradiction.
- **Price paid:** space and time are $O(b^d)$; here 500 states were stored for depth 8 – the main reason BFS does not scale to deep problems such as the 15-puzzle.